

Chaînes de Markov cachées et application en finance

Sous la supervision de *Joseph Rynkiewicz et Jean-Marc Bardet*.

Par Marius Requillart et Maxime Delpech.

2026

SOMMAIRE

I) Introduction

II) Définition et propriétés

- 2.1) Définitions et propriétés d'une chaîne de Markov
 - 2.1.1) Définitions
 - 2.1.2) Propriétés
- 2.2) Définition d'une chaîne de Markov cachée.
 - 2.2.1) Variables aléatoires observables et cachées.
 - 2.2.2) Lois jointes et vraisemblance.
 - 2.2.3) Exemple global d'une chaîne de Markov cachée.
 - 2.2.4) Remarques sur les observations.

III) Trois grandes problématiques des HMM

- 3.1) Estimation de la vraisemblance et algorithme Forward
- 3.2) Décodage et algorithme de Viterbi
- 3.3) Estimation et Algorithme de Baum–Welch

IV) Limites des algorithmes et Analyse de la qualité des estimations

- 4.1) Limites et Mesure de la qualité de la vraisemblance avec l'algorithme Forward
 - 4.1.1) Limites de l'algorithme de Forward
 - 4.1.2) Interprétation de la log-vraisemblance
 - 4.1.3) Comparaison entre modèles
 - 4.1.4) Évaluation hors échantillon
- 4.2) Mesure de la qualité du décodage par Viterbi
 - 4.2.1) Difficulté de l'évaluation sur données réelles
 - 4.2.2) Interprétation économique des régimes détectés
 - 4.2.3) Comparaison avec le posterior decoding
- 4.3) Mesure de la qualité des estimations de Baum–Welch
 - 4.3.1) Convergence de la log-vraisemblance
 - 4.3.2) Stabilité des paramètres estimés
 - 4.3.3) Interprétation économique des régimes
 - 4.3.4) Validation hors échantillon

V) Simulation d'un HMM et application à des données financières

- 5.1) Jeu de données simulé
- 5.2) Application des algorithmes sur données simulées
- 5.3) Interprétation des résultats obtenus sur les données simulées
 - 5.3.1) Analyse du problème de Forward
 - 5.3.2) Analyse du problème de Viterbi
 - 5.3.3) Analyse du problème de Baum–Welch
- 5.4) Détection de régimes de volatilité sur le VIX
- 5.5) Analyse de la qualité de l'estimation Baum–Welch sur le VIX

V) Références

1 Introduction

Les séries temporelles financières occupent une place centrale en finance quantitative, car elles permettent d'analyser l'évolution des prix d'actifs, des rendements, de la volatilité ou encore des volumes échangés au cours du temps. Cependant, ces séries présentent souvent des comportements complexes : elles sont bruitées, irrégulières, parfois instables, et peuvent être marquées par des changements soudains de dynamique. Une même série financière peut ainsi alterner entre des périodes relativement calmes et des périodes de forte incertitude, ce qui rend son analyse particulièrement délicate.

Les chaînes de Markov cachées, ou Hidden Markov Models, constituent un cadre probabiliste adapté pour modéliser ce type de phénomènes. Leur principe repose sur l'existence d'un état latent, non directement observable, qui évolue au cours du temps selon une chaîne de Markov. Les observations disponibles, comme les rendements financiers, sont alors supposées dépendre de cet état caché.

L'intérêt de ces modèles réside dans leur capacité à faire apparaître une structure cachée derrière les données observées. Ils permettent non seulement de détecter des changements de régime, mais aussi de donner une interprétation économique et financière aux résultats obtenus. Les états cachés identifiés peuvent par exemple être associés à des phases de croissance, de stress financier, de forte incertitude ou de retour à une situation plus stable. Ainsi, l'analyse ne se limite pas à une classification statistique des observations : elle peut aussi aider à relier les régimes détectés à des événements de marché, à des changements d'anticipations ou à des variations du niveau de risque.

Dans ce travail, nous étudierons les chaînes de Markov cachées et leurs applications aux séries temporelles financières. Nous présenterons d'abord le cadre théorique de ces modèles, avant d'aborder les principaux algorithmes d'inférence et d'estimation, notamment les algorithmes de Viterbi et de Baum-Welch. Nous analyserons ensuite leur utilisation pour identifier différents régimes de marché et mieux comprendre l'évolution des séries financières.

2 Définitions et propriétés

Toutes les variables aléatoires de ce travail sont définies sur l'espace probabilisé $(\Omega, \mathcal{F}, \mathbb{P})$.

2.1 Définitions et propriétés d'une chaîne de Markov

2.1.1 Définition

Définition d'une chaîne de Markov :

Soit E un ensemble fini ou dénombrable, appelé *espace d'états*. Soit $(X_n)_{n \geq 0}$ un *processus* à valeur dans E . On dit que $(X_n)_{n \geq 0}$ est une *chaîne de Markov* d'ordre 1, si :

Pour tout $n \geq 0$, pour tout $y \in E$, pour tout $x_0, \dots, x_{n-1}, x \in E$ tel que $\mathbb{P}(X_0 = x_0, \dots, X_{n-1} = y, X_n = x) > 0$, on a :

$$\mathbb{P}(X_{n+1} = y \mid X_n = x, X_{n-1} = x_{n-1}, \dots, X_0 = x_0) = \mathbb{P}(X_{n+1} = y \mid X_n = x).$$

On pose π_0 la distribution initiale, i.e., la loi de X_0 .

De plus, s'il existe $A : E \times E \rightarrow [0, 1]$ tel que $(x, y) \mapsto a_{x,y} = \mathbb{P}(X_{n+1} = y \mid X_n = x)$, pour tout $n \geq 0$, pour tout $x, y \in E$ tel que $\mathbb{P}(X_n = x) > 0$. On dit alors que $(X_n)_{n \geq 0}$ est une chaîne de Markov d'ordre 1 *homogène*. Dans ce cas, A est appelé *matrice de transition* du processus $(X_n)_{n \geq 0}$, avec $a_{x,y} > 0$ et $\sum_{y \in E} a_{x,y} = 1$, pour tout $x, y \in E$.

Dans ce travail, nous ne considérerons qu'uniquement les chaînes de Markov d'ordre 1 homogènes, c'est pourquoi nous les appellerons juste chaînes de Markov.

2.1.2 Propriétés

• Loi d'une trajectoire markovienne :

Soit $(X_n)_{n \geq 0}$ une chaîne de Markov de matrice de transition P et de distribution initiale π_0 . On a, Pour tout $n \geq 0$, pour tout $x_0, \dots, x_n \in E$,

$$\mathbb{P}(X_0 = x_0, \dots, X_n = x_n) = \pi_0(x_0) \mathbb{P}(x_0, x_1) \dots \mathbb{P}(x_{n-1}, x_n)$$

i.e. la loi d'une chaîne de Markov est entièrement caractérisée par sa distribution initiale et sa matrice de transition.

• Propriété de Markov :

Soit $n \geq 0$, π_0 une mesure de probabilité sur E et $x \in E$ tel que $\mathbb{P}_{\pi_0}(X_n = x) > 0$.

Alors, pour tout $A \in E^n$, $B \in E^m$, $m \geq 0$, on a :

$$\mathbb{P}_{\pi_0}((X_0, \dots, X_{n-1}) \in A, (X_{n+1}, \dots, X_{n+m}) \in B \mid X_n = x) = \mathbb{P}_{\pi_0}((X_0, \dots, X_{n-1}) \in A \mid X_n = x) \\ \times \mathbb{P}_{\pi_0}((X_{n+1}, \dots, X_{n+m}) \in B \mid X_n = x).$$

i.e. le passé et le futur d'un moment n d'une chaîne de Markov sont indépendants entre eux.

Quelques propriétés utiles :

Pour tout $x, y \in E$, on a :

$$\mathbb{P}_x(X_n = y) = P^n(x, y).$$

De plus, pour toute mesure de probabilité μ sur E , et pour tout $y \in E$, on a :

$$\mathbb{P}_\mu(X_n = y) = \sum_{x \in E} \mu(x) P^n(x, y) = (\mu P^n)(y).$$

2.2 Définition d'une chaîne de Markov cachée

Les chaînes de Markov cachées (HMM) sont des modèles statistiques utilisés dans une situation où on a des états latents qui ne sont pas directement observables. Les données observables sont générées par ces états suivant une loi de probabilité conditionnelle.

2.2.1 Variables aléatoires observables et cachées

Définition. Les états cachés sont modélisés par un processus de Markov $(X_t)_{t \in \mathbb{N}}$ à valeurs dans un ensemble fini $E = \{1, \dots, N\}$. On note $A = (a_{ij})$ la matrice de transition définie par :

$$a_{ij} = \mathbb{P}(X_{t+1} = j \mid X_t = i).$$

Définition. Les observations sont modélisées par un processus $(Y_t)_{t \in \mathbb{N}}$ à valeurs dans un ensemble O . Conditionnellement aux états cachés, les variables (Y_t) sont indépendantes et vérifient :

$$\mathbb{P}(Y_t = y \mid X_t = i) = b_i(y),$$

où $B = (b_i(y))$ est la matrice (ou famille) d'émission.

Définition. Un modèle de chaîne de Markov cachée est défini par le triplet (A, B, π) où :

- A est la matrice de transition des états cachés,
- B est la loi d'émission des observations,
- π est la distribution initiale, avec $\pi_i = \mathbb{P}(X_1 = i)$.

2.2.2 Loïs jointes et vraisemblance

Propriété : Soit $X = (X_1, \dots, X_T)$ la séquence d'états cachés et $Y = (Y_1, \dots, Y_T)$ la séquence d'observations, la loi des observations conditionnellement à la séquence d'états cachés est :

$$\mathbb{P}(Y | X) = \prod_{t=1}^T \mathbb{P}(Y_t | X_t)$$

Preuve :

$$\mathbb{P}(Y | X) = \prod_{t=1}^T \mathbb{P}(Y_t | X_1, \dots, X_T, Y_1, \dots, Y_{t-1}) = \prod_{t=1}^T \mathbb{P}(Y_t | X_t)$$

Propriété : Soit $X = (X_1, \dots, X_T)$ la séquence d'états cachés et $Y = (Y_1, \dots, Y_T)$ la séquence d'observations, la loi jointe des observations et des états cachés est donnée par :

$$\mathbb{P}(Y, X) = \pi_{X_1} \prod_{t=2}^T \mathbb{P}(X_t | X_{t-1}) \prod_{t=1}^T \mathbb{P}(Y_t | X_t)$$

Preuve :

$$\begin{aligned} \mathbb{P}(X, Y) &= \mathbb{P}(X_1, Y_1) \prod_{t=2}^T \mathbb{P}(X_t, Y_t | X_1, \dots, X_{t-1}, Y_1, \dots, Y_{t-2}) \\ &= \mathbb{P}(X_1) \mathbb{P}(Y_1 | X_1) \prod_{t=2}^T \mathbb{P}(X_t | X_1, \dots, X_{t-1}, Y_1, \dots, Y_{t-2}) \mathbb{P}(Y_t | X_1, \dots, X_{t-1}, Y_1, \dots, Y_{t-2}) \\ &= \mathbb{P}(X_1) \mathbb{P}(Y_1 | X_1) \prod_{t=2}^T \mathbb{P}(X_t | X_{t-1}) \mathbb{P}(Y_t | X_t) \\ &= \mathbb{P}(X_1) \prod_{t=2}^T \mathbb{P}(X_t | X_{t-1}) \prod_{t=1}^T \mathbb{P}(Y_t | X_t) \\ &= \pi_{X_1} \prod_{t=2}^T \mathbb{P}(X_t | X_{t-1}) \prod_{t=1}^T \mathbb{P}(Y_t | X_t) \end{aligned}$$

Remarque : La loi des observations à chaque instant t dépend de l'état caché à cet instant et pas des états d'avant.

Exemple : Soit un modèle de chaîne de Markov cachée avec deux états A et B . En supposant que les observations sont gaussiennes conditionnellement à l'état caché, on a :

$$\mathcal{L}(Y_t | X_t = A) \sim \mathcal{N}(\mu_A, \sigma_A^2)$$

$$\mathcal{L}(Y_t | X_t = B) \sim \mathcal{N}(\mu_B, \sigma_B^2)$$

Remarque : X_t agit comme un sélecteur de distribution pour Y_t .

2.2.3 Exemple global d'une chaîne de Markov cachée

Prenons un exemple simple d'une chaîne de Markov cachée avec deux états C et V (pour "calme" et "volatile") et des observations gaussiennes conditionnelles à ces états.

Soit $E = \{C, V\}$ l'ensemble d'états, on note π la distribution initiale, par exemple $\pi_0 = (0.5, 0.5)$, indiquant que le processus commence avec une probabilité égale dans les états C et V .

La matrice de transition A peut être définie comme suit :

$$A = \begin{pmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{pmatrix}$$

Avec, $a_{ij} = \mathbb{P}(X_{t+1} = j \mid X_t = i)$.

La famille de densités paramétriques B peut être définie par les distributions gaussiennes conditionnelles à chaque état :

$$B = \begin{cases} \mathcal{L}(Y_t \mid X_t = C) \sim \mathcal{N}(\mu_C, \sigma_C^2) \\ \mathcal{L}(Y_t \mid X_t = V) \sim \mathcal{N}(\mu_V, \sigma_V^2) \end{cases}$$

NB : La matrice de transition A n'est pas connue dans la plupart des applications, elle doit être estimée à partir des données d'observations. Par exemple avec l'algorithme de Baum-Welch que nous verrons plus tard.

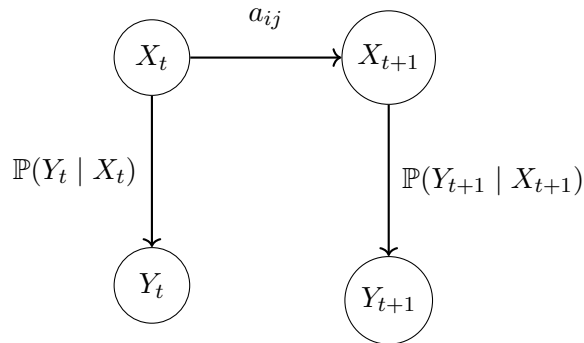


FIGURE 1 – Modèle de Markov caché (HMM)

2.2.4 Remarques sur les observations

Propriété : La probabilité totale des observations $(Y_t)_t$ est donnée par :

$$\mathbb{P}(Y) = \sum_X \mathbb{P}(Y, X) = \sum \mathbb{P}(Y \mid X) \mathbb{P}(X)$$

Propriété : Dans un HMM non dégénéré, le processus d'observations $(Y_t)_t$ n'est pas un processus de Markov, même si les états cachés $(X_t)_t$ le sont.

Preuve :

Si le processus d'observations $(Y_t)_t$ était markovien, alors on aurait

$$\mathbb{P}(Y_{n+1} \mid Y_1, \dots, Y_n) = \mathbb{P}(Y_{n+1} \mid Y_n).$$

Or, pour K états cachés, on a

$$\begin{aligned} \mathbb{P}(Y_{n+1} \mid Y_1, \dots, Y_n) &= \sum_{i=1}^K \mathbb{P}(Y_{n+1}, X_{n+1} = i \mid Y_1, \dots, Y_n) \\ &= \sum_{i=1}^K \mathbb{P}(Y_{n+1} \mid X_{n+1} = i, Y_1, \dots, Y_n) \mathbb{P}(X_{n+1} = i \mid Y_1, \dots, Y_n) \\ &= \sum_{i=1}^K \mathbb{P}(Y_{n+1} \mid X_{n+1} = i) \mathbb{P}(X_{n+1} = i \mid Y_1, \dots, Y_n). \end{aligned}$$

Or,

$$\begin{aligned} \mathbb{P}(X_{n+1} = i \mid Y_1, \dots, Y_n) &= \sum_{j=1}^K \mathbb{P}(X_{n+1} = i, X_n = j \mid Y_1, \dots, Y_n) \\ &= \sum_{j=1}^K \mathbb{P}(X_{n+1} = i \mid X_n = j, Y_1, \dots, Y_n) \mathbb{P}(X_n = j \mid Y_1, \dots, Y_n) \\ &= \sum_{j=1}^K \mathbb{P}(X_{n+1} = i \mid X_n = j) \mathbb{P}(X_n = j \mid Y_1, \dots, Y_n) \\ &= \sum_{j=1}^K a_{ji} \mathbb{P}(X_n = j \mid Y_1, \dots, Y_n). \end{aligned}$$

et,

$$\begin{aligned}
\mathbb{P}(X_n = j \mid Y_1, \dots, Y_n) &= \mathbb{P}(X_n = j, Y_1 \dots Y_n) / \mathbb{P}(Y_1 \dots Y_n) \\
&= \frac{\mathbb{P}(Y_n \mid X_n = j, Y_1 \dots Y_{n-1}) \mathbb{P}(X_n = j, Y_1 \dots Y_{n-1})}{\mathbb{P}(Y_1 \dots Y_n)} \\
&= \frac{\mathbb{P}(Y_n \mid X_n = j) \mathbb{P}(X_n = j \mid Y_1 \dots Y_{n-1}) \mathbb{P}(Y_1 \dots Y_{n-1})}{\mathbb{P}(Y_n \mid Y_1 \dots Y_{n-1}) \mathbb{P}(Y_1 \dots Y_{n-1})} \\
&= \frac{\mathbb{P}(Y_n \mid X_n = j) \mathbb{P}(X_n = j \mid Y_1 \dots Y_{n-1})}{\mathbb{P}(Y_n \mid Y_1 \dots Y_{n-1})}.
\end{aligned}$$

On a donc :

$$\mathbb{P}(Y_{n+1} \mid Y_1 \dots Y_n) = \sum_{i=1}^K \sum_{j=1}^K a_{ji} \frac{\mathbb{P}(Y_{n+1} \mid X_{n+1} = i) \mathbb{P}(Y_n \mid X_n = j) \mathbb{P}(X_n = j \mid Y_1 \dots Y_{n-1})}{\mathbb{P}(Y_n \mid Y_1 \dots Y_{n-1})}.$$

On procède de la même manière pour calculer $\mathbb{P}(X_n = j \mid Y_1 \dots Y_{n-1})$ ce qui va descendre le degré de dépendance à Y_{n-1} . On continue ainsi de suite de manière récursive jusqu'à ce que le terme de dépendance à Y_1 soit atteint.

Ainsi, le terme

$$\mathbb{P}(X_{n+1} = i \mid Y_1, \dots, Y_n)$$

dépend en général de tout le passé observé $(Y_1, \dots, Y_n)$, et pas seulement de Y_n .

Donc, en général,

$$\mathbb{P}(Y_{n+1} \mid Y_1, \dots, Y_n) \neq \mathbb{P}(Y_{n+1} \mid Y_n),$$

ce qui montre que le processus d'observations (Y_t) n'est pas markovien.

3 Trois grandes problématiques des HMM

3.1 Estimation de la vraisemblance et algorithme Forward

Problématique : Comment calculer efficacement la vraisemblance $\mathbb{P}(Y)$ d'une séquence d'observations Y donnée par un modèle de HMM (A, B, π) ?

On suppose que la matrice de transition A , la matrice d'émission B ainsi que la distribution initiale π sont connues.

On sait que la vraisemblance de Y est donnée par :

$$\mathbb{P}(Y) = \sum_X \mathbb{P}(Y, X) = \sum_X \mathbb{P}(Y | X) \mathbb{P}(X).$$

Cependant, pour N états cachés et une séquence d'observations de longueur T , le nombre de séquences cachées possibles est de N^T , ce qui rend le calcul direct d'une complexité exponentielle si on l'effectue de manière naïve.

La solution optimale est d'utiliser l'algorithme de forward qui permet de calculer $\mathbb{P}(Y)$ avec une complexité de $O(N^2T)$.

Cet algorithme repose sur le concept de programmation dynamique et utilise une variable intermédiaire $\alpha_t(i)$ qui représente la probabilité d'observer la séquence d'observations $y_1 \dots y_t$ et d'être dans l'état caché i à l'instant t .

Formellement, on définit $\alpha_t(i)$ comme :

$$\alpha_t(i) = \mathbb{P}(y_1, \dots, y_t, X_t = i).$$

Algorithm 1 Algorithme Forward

Require: Matrice de transition $A = (a_{ij})_{1 \leq i, j \leq N}$, probabilités d'émission $B = (b_i(y))_{1 \leq i \leq N}$, distribution initiale $\pi = (\pi_i)_{1 \leq i \leq N}$, suite d'observations $Y = (y_1, \dots, y_T)$

Ensure: Vraisemblance $\mathbb{P}(Y)$

Initialisation

```

1: for  $i = 1, \dots, N$  do
2:    $\alpha_1(i) \leftarrow \pi_i b_i(y_1)$ 
3: end for
```

Récursion

```

4: for  $t = 2, \dots, T$  do
5:   for  $j = 1, \dots, N$  do
6:      $\alpha_t(j) \leftarrow b_j(y_t) \sum_{i=1}^N \alpha_{t-1}(i) a_{ij}$ 
7:   end for
8: end for
```

Terminaison

```

9:  $\mathbb{P}(Y) \leftarrow \sum_{i=1}^N \alpha_T(i)$ 
10: return  $\mathbb{P}(Y)$ 
```

avec a_{ij} la probabilité de transition de l'état i à l'état j , $b_j(y_t)$ la probabilité d'observer $Y_t = y_t$ conditionnellement à être dans l'état j à l'instant t , et π_i la probabilité initiale d'être dans l'état i . L'algorithme de forward permet ainsi de calculer efficacement la vraisemblance d'une séquence d'observations donnée, en évitant le calcul direct de toutes les séquences d'états cachés possibles.

Analyse de la complexité :

Lors de l'initialisation, on effectue N opérations de complexité $O(1)$ pour calculer $\alpha_1(i)$ pour $i = 1, \dots, N$, soit une complexité de $O(N)$.

Lors de la récursion, pour chaque instant t de 2 à T , on effectue N opérations pour calculer $\alpha_t(j)$ pour $j = 1, \dots, N$. Chaque calcul de $\alpha_t(j)$ nécessite de faire une somme de N termes, ce qui correspond à une complexité de $O(N^2)$ pour chaque instant t . Ainsi la complexité totale de la récursion est de $O(N^2T)$.

Lors de la terminaison, on effectue une somme de N termes pour calculer $\mathbb{P}(Y)$, ce qui correspond à une complexité de $O(N)$.

En combinant ces différentes étapes, on obtient une complexité totale de l'algorithme de $O(N^2T)$, ce qui est beaucoup plus efficace que la complexité exponentielle $O(N^T)$ du calcul brut.

Pour plus de clarté, on peut présenter l'algorithme de forward sous forme de treillis :

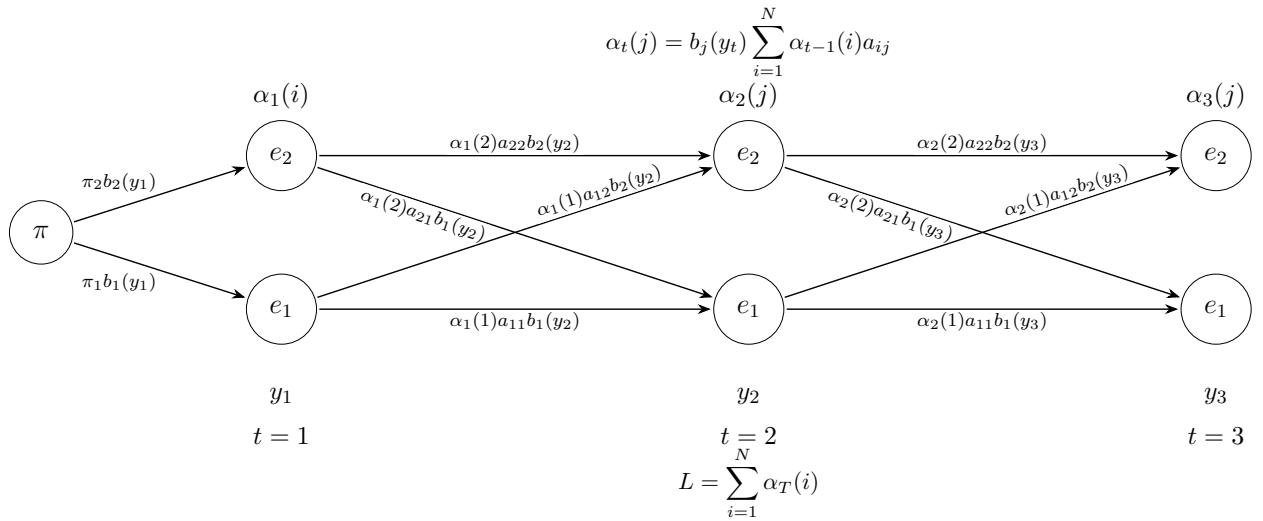


FIGURE 2 – Treillis de l'algorithme Forward

3.2 Décodage et algorithme de Viterbi

Considérons un nouveau problème : le décodage d'une suite d'états d'après une série d'observations $Y_1, \dots, Y_T$. On essaye de découvrir la partie cachée du modèle.

i.e. Découvrir a posteriori la “vraie” séquence d’états $X = (X_1, \dots, X_T)$ qui s’est produite pour générer une série d’observations, sachant la distribution initiale, la matrice de transition ainsi que les probabilités d’émission.

Il paraît évident qu’il n’y a pas toujours de “vraie” séquence d’états, nous allons devoir instaurer des critères d’optimalité pour régler notre problème.

Une première approche qu’on pourrait qualifier de “Greedy” serait de choisir, à chaque instant t , l’état X_t tel que la probabilité d’observer Y_t soit la plus grande. Ce critère d’optimalité maximise l’espérance du nombre d’états correctement décryptés un à un.

Cependant, cela peut amener plusieurs problèmes avec la séquence d’états qui en résulte. Comme le critère local ignore la matrice de transition, il peut proposer une transition de probabilité nulle. Or, rien ne l’interdit dans notre critère, donc cette transition pourrait apparaître dans le résultat alors que celui-ci ne serait ainsi même pas réalisable.

Il va donc falloir trouver un critère prenant en compte la probabilité de réalisation d’une séquence d’états. On pourrait ainsi vouloir maximiser l’espérance du nombre de paires correctes d’états découverts, ou de triplets, etc. Cependant, le critère le plus utilisé est le critère de trouver l’unique séquence d’états qui maximise la probabilité de voir nos observations sachant la distribution initiale, la matrice de transition ainsi que les probabilités d’émission.

Le problème étant exponentiellement complexe, la solution est d’utiliser un algorithme de programmation dynamique nommé : Algorithme de Viterbi.

3.2.1 L’algorithme de Viterbi

Premièrement, on remarque que maximiser $\mathbb{P}(X \mid Y, \lambda)$ est équivalent à maximiser $\mathbb{P}(X, y \mid \lambda)$ car les observations sont fixées.

Afin de trouver la meilleure séquence d’états $X = \{X_1, \dots, X_T\}$, pour la séquence d’observations $y = \{y_1, \dots, y_T\}$, on définit la quantité :

$$\delta_t(i) = \max_{X_1, \dots, X_{t-1}} \mathbb{P}(X_1, X_2, \dots, X_t = i, y_1, y_2, \dots, y_t \mid \lambda)$$

i.e. $\delta_t(i)$ est la probabilité du chemin le plus probable se terminant par l’état i au moment t .

Avec,

$$a_{ij} = \mathbb{P}(X_{t+1} = j \mid X_t = i)$$

et, si les observations sont discrètes,

$$b_j(y_t) = \mathbb{P}(Y_t = y_t \mid X_t = j)$$

ou, si les observations sont continues,

$$b_j(y_t) = f_{Y_t|X_t=j}(y_t)$$

Ainsi, par récurrence, on a que :

$$\delta_{t+1}(j) = \left[\max_{1 \leq i \leq N} \delta_t(i) a_{ij} \right] b_j(y_{t+1})$$

Problème : En testant naïvement toutes les suites possibles d'états cachés, il y a N^T séquences possibles. Et pour calculer la probabilité de chaque séquence, il faut environ T opérations.

Ainsi on obtient une complexité de :

$$O(TN^T)$$

C'est exponentiel, donc inutilisable dès que T ou N devient grand.

Afin de limiter cette complexité, l'algorithme de Viterbi repose sur le principe du treillis.

Principe du Treillis :

Structure en colonne où :

- chaque colonne correspond à un instant t
- chaque ligne correspond à un état caché.
- chaque nœud (t, j) représente le fait d'être dans l'état j au temps t .
- chaque arête entre $(t-1, i)$ et (t, j) représente une transition possible, pondérée par la proba a_{ij} .
- chaque nœud est également associé à la proba d'émission $b_j(y_t)$.

Ainsi, le problème de décodage revient à trouver le meilleur chemin dans ce treillis, depuis la première colonne jusqu'à la dernière. Chaque chemin correspond à une séquence d'états cachés possibles. Le rôle de l'algorithme de Viterbi est précisément de trouver le chemin de probabilité maximale :

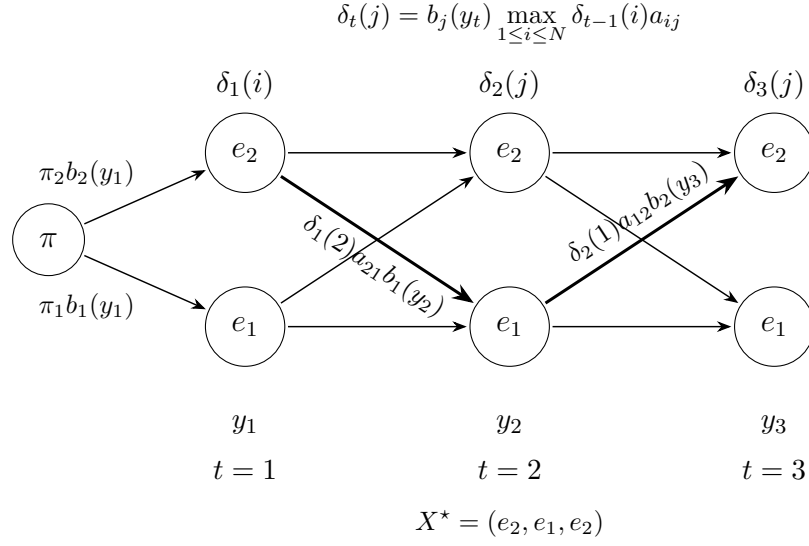


FIGURE 3 – Treillis de l'algorithme de Viterbi

L'intérêt du treillis est qu'il met en évidence une propriété fondamentale : pour retrouver la séquence d'états, il faut garder en mémoire l'argument qui maximise la récurrence. Nous allons le faire via la liste $\psi_t(j)$.

On peut donc suivre l'algorithme suivant :

Algorithm 2 Algorithmme de Viterbi

Require: Matrice de transition $A = (a_{ij})_{1 \leq i, j \leq N}$, probabilités d'émission $B = (b_i(y))_{1 \leq i \leq N}$, distribution initiale $\pi = (\pi_i)_{1 \leq i \leq N}$, suite d'observations $Y = (y_1, \dots, y_T)$

Ensure: Chemin d'états le plus probable $X^* = (X_1^*, \dots, X_T^*)$ et probabilité associée P^*

Initialisation

```
1: for  $i = 1, \dots, N$  do  
2:    $\delta_1(i) \leftarrow \pi_i b_i(y_1)$   
3:    $\psi_1(i) \leftarrow 0$   
4: end for
```

Récurrance

```
5: for  $t = 2, \dots, T$  do  
6:   for  $j = 1, \dots, N$  do  
7:      $\delta_t(j) \leftarrow \left[ \max_{1 \leq i \leq N} \delta_{t-1}(i) a_{ij} \right] b_j(y_t)$   
8:      $\psi_t(j) \leftarrow \arg \max_{1 \leq i \leq N} \delta_{t-1}(i) a_{ij}$   
9:   end for  
10: end for
```

Fin

```
11:  $P^* \leftarrow \max_{1 \leq i \leq N} \delta_T(i)$   
12:  $X_T^* \leftarrow \arg \max_{1 \leq i \leq N} \delta_T(i)$ 
```

Retraçage du chemin

```
13: for  $t = T - 1, T - 2, \dots, 1$  do  
14:    $X_t^* \leftarrow \psi_{t+1}(X_{t+1}^*)$   
15: end for  
16: return  $X^*, P^*$ 
```

On obtient ainsi avec $X_1^*, \dots, X_T^*$ notre séquence d'états recherchée.

Analyse de la complexité :

- À chaque instant, on calcule une valeur pour chacun des N états.
- Pour chaque état, on doit évaluer la probabilité de venir depuis chacun des états.

Ainsi, à chaque itération, on a une complexité de $O(N^2)$.

Il y a T instants, on obtient donc une complexité temporelle totale de :

$$O(TN^2)$$

3.3 Estimation et Algorithme de Baum–Welch

Ce dernier problème est celui qui nous intéressera le plus dans les applications économiques. Nous allons chercher à estimer l'ensemble des paramètres de notre modèle, la distribution initiale, les probabilités de changement d'états ainsi que les paramètres des différentes lois des observations, et tout ça uniquement à partir d'une série d'observations. C'est-à-dire qu'on cherche le modèle qui maximise la probabilité de voir ce qu'on a observé.

i.e.

$$\operatorname{argmax}_{(A,B,\pi)} \mathbb{P}(Y \mid (A, B, \pi))$$

Pour cela, nous pouvons utiliser un algorithme introduit par Baum–Welch qui est en fait un cas particulier d'utilisation de l'algorithme Expectation Maximisation (EM).

Avant toute chose, il est important de noter que cette méthode itérative permet d'approximer un optimum local à notre problème.

Afin de pouvoir trouver les A , B et π de notre problème, nous devons définir les variables avec lesquelles nous allons travailler :

- $\xi_t(i, j) :=$ la probabilité d'être dans l'état i à l'instant t et dans l'état j à l'instant $t + 1$ sachant le modèle (A, B, π) et la série d'observations Y .

$$\forall t \in \{1, \dots, T - 1\}, \quad \forall i, j \in E,$$

$$\begin{aligned} \xi_t(i, j) &= \mathbb{P}(X_t = i, X_{t+1} = j \mid Y, (A, B, \pi)) \\ &= \frac{\mathbb{P}(X_t = i, X_{t+1} = j, Y \mid (A, B, \pi))}{\mathbb{P}(Y \mid A, B, \pi)} \\ &= \frac{\alpha_t(i) a_{ij} b_j(y_{t+1}) \beta_{t+1}(j)}{\mathbb{P}(Y \mid A, B, \pi)} \\ &= \frac{\alpha_t(i) a_{ij} b_j(y_{t+1}) \beta_{t+1}(j)}{\sum_{k \in E} \sum_{\ell \in E} \alpha_t(k) a_{k\ell} b_\ell(y_{t+1}) \beta_{t+1}(\ell)} \end{aligned}$$

- $\gamma_t(i) :=$ la probabilité d'être dans l'état i à l'instant t , sachant la série d'observations et le modèle (A, B, π) .

$$\forall t \in \{1, \dots, T-1\}, \quad \forall i \in E,$$

$$\begin{aligned} \gamma_t(i) &= \mathbb{P}(X_t = i \mid Y, (A, B, \pi)) \\ &= \sum_{j \in E} \mathbb{P}(X_t = i, X_{t+1} = j \mid Y, (A, B, \pi)) \\ &= \sum_{j \in E} \xi_t(i, j) \end{aligned}$$

On peut remarquer qu'en sommant les $\gamma_t(i)$ par rapport aux instants t , on obtient le nombre de fois où l'état i est visité, ou de manière équivalente, l'espérance du nombre de transitions depuis i .

De la même manière, la somme sur les t des $\xi_t(i, j)$ donne l'espérance du nombre de transitions de l'état i vers l'état j .

i.e.

$$\begin{aligned} \sum_{t=1}^T \gamma_t(i) &= \text{espérance du nombre de transitions depuis } i \\ \sum_{t=1}^{T-1} \xi_t(i, j) &= \text{espérance du nombre de transitions de } i \text{ vers } j \end{aligned}$$

Grâce à cette remarque, des estimateurs naturels de π , A , B apparaissent :

$$\forall i, j \in E,$$

$$\pi'_i := \text{probabilité d'être dans l'état } i \text{ au moment } 1 = \gamma_1(i)$$

$$a'_{ij} := \frac{\text{espérance du nombre de transitions de l'état } i \text{ à } j}{\text{espérance du nombre de transitions depuis l'état } i} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

$$\begin{aligned} b'_j(y) &:= \frac{\text{espérance du nombre d'observations de } y \text{ étant dans l'état } j}{\text{espérance du nombre de fois où on est dans l'état } j} \\ &= \frac{\sum_{t=1}^T \gamma_t(j) \mathbf{1}_{\{y_t=y\}}}{\sum_{t=1}^T \gamma_t(j)} \end{aligned}$$

Dans le cas où les observations sont continues, alors on estime les paramètres avec des estimateurs naturels issus de ces quantités.

Par exemple pour des émissions gaussiennes, on a :

$$\mu_j = \frac{\sum_{t=1}^T \gamma_t(j) y_t}{\sum_{t=1}^T \gamma_t(j)}$$

et

$$\sigma_j^2 = \frac{\sum_{t=1}^T \gamma_t(j) (y_t - \mu_j)^2}{\sum_{t=1}^T \gamma_t(j)}$$

On peut donc en déduire cet algorithme :

Algorithm 3 Algorithme de Baum–Welch

Require: Suite d'observations $Y = (y_1, \dots, y_T)$,

- 1: nombre d'états N ,
- 2: paramètres initiaux $\lambda^{(0)} = (A^{(0)}, B^{(0)}, \pi^{(0)})$,
- 3: seuil de convergence ε

Ensure: Paramètres estimés $\hat{\lambda} = (\hat{A}, \hat{B}, \hat{\pi})$

- 4: Initialiser $m \leftarrow 0$
- 5: **repeat**

Étape E : estimation des variables cachées

- 6: Pour $i = 1, \dots, N$:

$$\alpha_1(i) = \pi_i^{(m)} b_i^{(m)}(y_1)$$

- 7: Pour $t = 2, \dots, T$ et $j = 1, \dots, N$:

$$\alpha_t(j) = b_j^{(m)}(y_t) \sum_{i=1}^N \alpha_{t-1}(i) a_{ij}^{(m)}$$

- 8: Pour $i = 1, \dots, N$:

$$\beta_T(i) = 1$$

- 9: Pour $t = T - 1, \dots, 1$ et $i = 1, \dots, N$:

$$\beta_t(i) = \sum_{j=1}^N a_{ij}^{(m)} b_j^{(m)}(y_{t+1}) \beta_{t+1}(j)$$

- 10: Pour $t = 1, \dots, T$ et $i = 1, \dots, N$:

$$\gamma_t(i) = \frac{\alpha_t(i) \beta_t(i)}{\sum_{k=1}^N \alpha_t(k) \beta_t(k)}$$

- 11: Pour $t = 1, \dots, T - 1$ et $i, j = 1, \dots, N$:

$$\xi_t(i, j) = \frac{\alpha_t(i) a_{ij}^{(m)} b_j^{(m)}(y_{t+1}) \beta_{t+1}(j)}{\sum_{k=1}^N \sum_{\ell=1}^N \alpha_t(k) a_{k\ell}^{(m)} b_{\ell}^{(m)}(y_{t+1}) \beta_{t+1}(\ell)}$$

Étape M : mise à jour des paramètres

- 12: Pour $i = 1, \dots, N$:

$$\pi_i^{(m+1)} = \gamma_1(i)$$

- 13: Pour $i, j = 1, \dots, N$:

$$a_{ij}^{(m+1)} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

- 14: Mettre à jour les paramètres d'émission $B^{(m+1)}$ avec des estimateurs naturels basés sur les $\gamma_t(i)$

- 15: $L^{(m+1)} \leftarrow \sum_{i=1}^N \alpha_T(i)$

- 16: $m \leftarrow m + 1$

- 17: **until** $|L^{(m)} - L^{(m-1)}| < \varepsilon$

- 18: **return** $\hat{\lambda} = \lambda^{(m)}$
-

Dans ce travail, nous ne traiterons pas du choix d'épsilon qui mériterait un travail à part entière, mais il est important de noter que ce choix peut avoir un impact significatif sur la convergence de l'algorithme. De plus, avec une seule série d'observations, on ne peut pas estimer de manière robuste la distribution initiale π , car elle ne concerne que l'état caché au premier instant ; toutefois, cela a peu d'impact sur l'utilisation finale du modèle, puisque l'influence de π disparaît progressivement au fil du temps au profit des transitions et des émissions estimées.

Analyse de la complexité :

- étape forward : pour chaque instant t , pour chaque état j , on somme sur tous les états précédents. Donc $O(TN^2)$.
- étape backward : idem que pour forward. Donc $O(TN^2)$.
- calcul des $\gamma_t(i)$: calcul utilisant $\alpha_t(i)$ et $\beta_t(i)$, sur chaque t et chaque i . Donc $O(TN)$.
- calcul des $\xi_t(i, j)$: calcul utilisant $\alpha_t(i)$ et $\beta_t(j)$ pour chaque t et chaque couple (i, j) . Donc $O(TN^2)$.
- mise à jour des π_i : on récupère la valeur de $\gamma_1(i)$ pour chaque i . Donc $O(N)$.
- mise à jour des a_{ij} : calcul avec une somme sur T pour chaque couple (i, j) . Donc $O(TN^2)$.
- mise à jour des $b_j(k)$: si on boucle naïvement sur tous les symboles k , les états j et les instants t , alors $O(TNM)$. Or on peut parcourir les observations efficacement et obtenir une complexité de $O(TN)$.

Ainsi, on obtient une complexité globale par itération de :

$$O(TN^2)$$

Si l'algorithme fait I itérations avant convergence, alors on a une complexité totale de :

$$O(ITN^2)$$

4 Limites des algorithmes et analyse de la qualité des estimations

4.1 Limites et Mesure de la qualité de la vraisemblance avec l'algorithme Forward

4.1.1 Limites de l'algorithme de forward

Une des principales limites de l'algorithme de forward vient de la quantité d'observations. En effet, plus la séquence d'observations est longue, plus la vraisemblance $\mathbb{P}(Y)$ devient extrêmement petite, ce qui peut entraîner des problèmes computationnels liés à la précision numérique. Pour pallier ce problème, on peut alors effectuer une modification de l'algorithme de forward en passant à la log-vraisemblance. Cela permet de transformer les produits en sommes, le tout avec des valeurs plus grandes et par conséquent plus faciles à manipuler numériquement.

De plus, si le nombre d'états cachés est très grand, l'algorithme de forward peut devenir extrêmement coûteux en temps de calcul, ce qui peut rendre son utilisation moins pertinente dans des applications avec un grand nombre d'états.

Aussi, l'algorithme Forward évalue la vraisemblance des observations, mais il ne dit pas directement si les régimes cachés estimés sont économiquement interprétables. Un modèle peut avoir une bonne vraisemblance tout en produisant des régimes difficiles à lire financièrement. Ainsi, la vraisemblance calculée par Forward doit être utilisée comme une mesure statistique importante, mais elle doit être complétée par l'analyse des régimes obtenus avec Viterbi ou avec les probabilités a posteriori, ainsi que par une validation hors échantillon.

L'algorithme Forward permet donc de calculer la probabilité d'observer une séquence donnée sous un modèle HMM donné. Si $Y_{1:T}$ désigne la série observée, avec T le nombre total d'observations, et si $\lambda = (\pi, A, B)$ désigne les paramètres du modèle, l'algorithme Forward calcule :

$$P(Y_{1:T} \mid \lambda).$$

En pratique, on utilise plutôt la log-vraisemblance :

$$\ell(\lambda) = \log P(Y_{1:T} \mid \lambda),$$

car la vraisemblance brute devient très petite lorsque T est grand.

4.1.2 Interprétation de la log-vraisemblance

La log-vraisemblance mesure la capacité du modèle à expliquer les observations. Plus elle est élevée, plus le modèle attribue une forte probabilité à la série observée. Elle constitue donc une première

mesure de qualité statistique du HMM.

Cependant, cette mesure doit être interprétée avec prudence. Un modèle plus complexe, par exemple avec davantage d'états cachés, peut obtenir une meilleure log-vraisemblance simplement parce qu'il possède plus de paramètres. Une bonne log-vraisemblance ne signifie donc pas nécessairement que les régimes estimés sont plus interprétables, ni que le modèle restera performant sur de nouvelles observations.

4.1.3 Comparaison entre modèles

Pour comparer plusieurs HMM, on peut utiliser la log-vraisemblance moyenne par observation :

$$\bar{\ell}(\lambda) = \frac{1}{T} \log P(Y_{1:T} \mid \lambda).$$

Cette normalisation permet de comparer des séries de longueurs différentes.

On peut aussi utiliser des critères pénalisés comme l'AIC et le BIC :

$$AIC = -2\ell(\lambda) + 2p,$$

$$BIC = -2\ell(\lambda) + p \log(T),$$

où p désigne le nombre de paramètres estimés. Ces critères pénalisent les modèles trop complexes. Dans ce cas, le meilleur modèle est celui qui minimise l'AIC ou le BIC.

4.1.4 Évaluation hors échantillon

Une bonne log-vraisemblance sur l'échantillon d'apprentissage ne suffit pas. Le modèle peut très bien expliquer les données passées tout en étant moins performant sur des données nouvelles. Pour une série temporelle financière, on découpe donc le jeu de données dans l'ordre chronologique : une première partie sert à l'apprentissage, puis la partie qui suit sert au test. Il faut donc évaluer la vraisemblance sur cette période de test, qui n'a pas servi à l'estimation.

Si le modèle $\hat{\lambda}_{\text{train}}$ est estimé sur $Y_{1:T_{\text{train}}}$, et si Y_{test} désigne les observations de la période test, on peut calculer :

$$\ell_{\text{test}} = \log P(Y_{\text{test}} \mid \hat{\lambda}_{\text{train}}).$$

On utilise souvent la log-vraisemblance moyenne de test :

$$\bar{\ell}_{\text{test}} = \frac{1}{T_{\text{test}}} \ell_{\text{test}},$$

ou encore la negative log predictive density :

$$NLDP = -\frac{1}{T_{\text{test}}} \ell_{\text{test}}.$$

Un *NLPD* faible indique que le modèle attribue une forte probabilité aux nouvelles observations. Il s'agit donc d'un bon indicateur pour vérifier si le modèle reste pertinent sur des données qui n'ont pas servi à l'estimation.

4.2 Mesure de la qualité du décodage par Viterbi

L'algorithme de Viterbi permet d'estimer la séquence d'états cachés la plus probable compte tenu des observations et des paramètres du modèle. Si $Y_{1:T}$ désigne la série observée, avec T le nombre total d'observations, et $\hat{\lambda}$ les paramètres estimés du modèle, Viterbi cherche le chemin

$$\hat{X}_{1:T}^V \in \arg \max_{x_{1:T}} P(X_{1:T} = x_{1:T} \mid Y_{1:T}, \hat{\lambda}).$$

Dans une application financière, ce chemin peut être interprété comme une chronologie des régimes de marché : chaque date est associée à un régime estimé, par exemple calme, volatil, stressé ou en reprise.

4.2.1 Difficulté de l'évaluation sur données réelles

La difficulté principale est que, sur données financières réelles, les vrais états cachés ne sont pas observés. On ne dispose donc pas d'une séquence vraie à laquelle comparer directement la séquence estimée. Il est alors difficile de calculer des mesures quantitatives classiques comme une accuracy, une matrice de confusion ou un taux d'erreur.

L'évaluation de Viterbi est donc surtout indirecte. Un chemin Viterbi très net n'est pas nécessairement le vrai chemin des états cachés : c'est seulement le chemin le plus probable sous le modèle estimé.

4.2.2 Interprétation économique des régimes détectés

Sur données réelles, un premier critère consiste à comparer les régimes détectés à des périodes financières connues. Par exemple, un régime interprété comme un régime de stress devrait apparaître davantage pendant des crises, des krachs ou des périodes de forte volatilité. À l'inverse, un régime calme devrait dominer pendant des périodes plus régulières.

Cette lecture reste qualitative, mais elle est importante. Si les états détectés ne correspondent à aucune dynamique financière claire, le décodage est difficile à exploiter.

On peut aussi vérifier la persistance des régimes. Un chemin qui change d'état presque à chaque date peut indiquer que le modèle capte surtout du bruit. Le chemin doit donc être assez stable pour être interprétable.

4.2.3 Comparaison avec le posterior decoding

Viterbi ne choisit pas l'état le plus probable à chaque date séparément. Il choisit le chemin globalement le plus probable, c'est-à-dire qu'il maximise :

$$P(X_{1:T} = x_{1:T} \mid Y_{1:T}, \hat{\lambda}).$$

Cette contrainte donne un chemin cohérent temporellement, mais elle peut conduire à choisir, à certaines dates, un état qui n'est pas celui ayant la plus forte probabilité marginale.

Le posterior decoding adopte une logique différente. Il calcule les probabilités a posteriori :

$$\hat{\gamma}_t(i) = P(X_t = i \mid Y_{1:T}, \hat{\lambda}),$$

puis choisit, à chaque date t , l'état marginalement le plus probable :

$$\hat{X}_t^P \in \arg \max_{i \in E} \hat{\gamma}_t(i).$$

Ainsi, le posterior decoding prend une décision locale date par date, tandis que Viterbi optimise une trajectoire complète.

Comparer les deux approches est intéressant, car elles ne répondent pas exactement à la même question. Si Viterbi et posterior decoding donnent des classifications proches, cela indique que les régimes sont bien identifiés par le modèle.

Un exemple simple de mesure d'écart est le taux de désaccord entre les deux décodages :

$$D_{VP} = \frac{1}{T} \sum_{t=1}^T \mathbf{1}_{\{\hat{X}_t^V \neq \hat{X}_t^P\}}.$$

Plus D_{VP} est proche de 0, plus les deux méthodes donnent des régimes similaires. Une valeur élevée indique au contraire que le choix des états est incertain ou dépend fortement de la méthode de décodage.

À l'inverse, si les deux méthodes donnent des résultats très différents, cela révèle une incertitude importante sur les états cachés. Le chemin Viterbi doit alors être interprété comme une trajectoire probable, et non comme une classification certaine.

4.3 Mesure de la qualité des estimations de Baum–Welch

Baum–Welch estime les paramètres d'une chaîne de Markov cachée à partir des seules observations $Y_{1:T}$. Si X_t désigne l'état caché à l'instant t , le modèle est donné par :

$$\lambda = (\pi, A, B)$$

avec

$$\pi_i = P(X_1 = i), \quad a_{ij} = P(X_{t+1} = j \mid X_t = i)$$

et B l'ensemble des lois d'émission $b_i(y)$, ou plus généralement $b_i(y; \theta_i)$ lorsque ces lois dépendent de paramètres θ_i . L'algorithme cherche alors

$$\hat{\lambda} \in \arg \max_{\lambda} P(Y_{1:T} \mid \lambda).$$

Comme il s'agit d'une méthode de type EM, la solution obtenue est en général un optimum local et pas nécessairement le meilleur modèle possible. La qualité des estimations doit donc être étudiée à travers la convergence, la stabilité des paramètres, l'interprétation économique et la performance hors échantillon.

4.3.1 Convergence de la log-vraisemblance

À l'itération k , on note $\ell^{(k)} = \log P(Y_{1:T} \mid \lambda^{(k)})$. Ici, T est la longueur de la série observée, c'est-à-dire le nombre total d'observations. Ainsi, les quantités divisées par T sont des gains moyens par observation.

Baum–Welch doit produire une log-vraisemblance non décroissante. On suit donc :

$$\Delta_k = \ell^{(k)} - \ell^{(k-1)}, \quad \bar{\Delta}_k = \frac{\Delta_k}{T}, \quad r_k = \frac{\Delta_k}{1 + |\ell^{(k-1)}|}.$$

Un critère d'arrêt raisonnable est $r_k < 10^{-6}$, ou bien $\bar{\Delta}_k < 10^{-6}$ à 10^{-5} , pendant plusieurs itérations consécutives. Ces seuils ne doivent pas être vus comme des règles absolues, mais comme des ordres de grandeur permettant de repérer le moment où l'amélioration devient négligeable.

Une convergence trop rapide peut être suspecte si l'algorithme s'arrête en moins de cinq itérations avec des paramètres proches de l'initialisation. Ce cas n'est pas forcément mauvais, mais il doit être comparé aux résultats obtenus avec d'autres initialisations. En lançant Baum–Welch M fois, on compare :

$$\frac{\ell_{\max}^* - \ell_m^*}{T}, \quad \ell_{\max}^* = \max_{1 \leq m \leq M} \ell_m^*.$$

Un écart de 10^{-3} à 10^{-2} par observation peut signaler une solution nettement moins bonne. À l'inverse, une convergence instable apparaît lorsque $\Delta_k < 0$. Une petite baisse numérique peut être tolérée si $\Delta_k/T > -10^{-8}$, mais des baisses plus fortes ou répétées indiquent un problème de normalisation, de sous-flux numérique ou d'implémentation.

4.3.2 Stabilité des paramètres estimés

Comme Baum–Welch dépend de l'initialisation, il faut répéter l'estimation avec plusieurs points de départ. Avant de comparer les résultats, on aligne les états afin de corriger le changement d'étiquette :

l'état 1 d'une estimation peut correspondre à l'état 2 d'une autre. Pour deux estimations (a) et (b), on peut mesurer :

$$D_\pi = \frac{1}{2} \sum_i \left| \hat{\pi}_i^{(a)} - \hat{\pi}_i^{(b)} \right|, \quad D_A = \frac{1}{N} \sum_i \frac{1}{2} \sum_j \left| \hat{a}_{ij}^{(a)} - \hat{a}_{ij}^{(b)} \right|.$$

On compare également la durée moyenne des régimes, pour $\hat{a}_{ii} < 1$:

$$\hat{d}_i = \frac{1}{1 - \hat{a}_{ii}}, \quad D_{d,i} = \left| \frac{\hat{d}_i^{(a)} - \hat{d}_i^{(b)}}{\hat{d}_i^{(b)}} \right|.$$

Deux estimations peuvent avoir des vraisemblances proches mais des régimes de persistance très différents.

Pour les lois d'émission discrètes, on peut utiliser :

$$D_B = \frac{1}{N} \sum_i \frac{1}{2} \sum_{y \in \mathcal{Y}} \left| \hat{b}_i^{(a)}(y) - \hat{b}_i^{(b)}(y) \right|.$$

Pour des lois continues de densités $\hat{f}_i^{(a)}$ et $\hat{f}_i^{(b)}$, on peut utiliser une divergence de Kullback–Leibler :

$$\text{KL}(\hat{f}_i^{(a)} \parallel \hat{f}_i^{(b)}) = \int \hat{f}_i^{(a)}(y) \log \left(\frac{\hat{f}_i^{(a)}(y)}{\hat{f}_i^{(b)}(y)} \right) dy.$$

Si les émissions sont paramétrées par θ_i , on compare les vecteurs $\hat{\theta}_i$, idéalement avec une distance normalisée afin de tenir compte des différences d'échelle entre les composantes.

4.3.3 Interprétation économique des régimes

Une estimation satisfaisante doit produire des régimes économiquement lisibles. Les lois d'émission associées aux états doivent décrire des comportements distincts. Si deux états ont des lois presque identiques, le modèle utilise probablement trop d'états ou ne sépare pas clairement les régimes. La matrice de transition doit aussi être cohérente : dans les séries financières, les coefficients diagonaux a_{ii} ne doivent pas être trop faibles, car les régimes de marché présentent souvent une certaine persistance.

Les probabilités a posteriori

$$\hat{\gamma}_t(i) = P(X_t = i \mid Y_{1:T}, \hat{\lambda})$$

permettent de vérifier à quelles dates chaque régime domine. Un régime interprétable doit correspondre à des périodes financières cohérentes, par exemple des périodes de stress, de calme ou de forte incertitude.

4.3.4 Validation hors échantillon

On parle aussi d'évaluation hors échantillon, de test hors échantillon, d'évaluation sur période de test ou encore d'évaluation prédictive sur données futures. Dans un contexte financier, le terme backtesting est également utilisé lorsque l'on teste une méthode sur une période passée qui n'a pas servi à l'estimation.

L'idée est d'estimer le modèle sur une période d'apprentissage puis de l'évaluer sur une période future non utilisée pour l'estimation. Si $\hat{\lambda}_{train}$ est estimé sur $Y_{1:T_{train}}$, on calcule :

$$\ell_{test} = \log P(Y_{T_{train}+1:T} | \hat{\lambda}_{train}), \quad NLPD = -\frac{1}{T_{test}} \ell_{test}.$$

Un $NLPD$ faible indique que le modèle attribue une forte probabilité aux nouvelles observations. Cette étape est importante, car une estimation peut bien expliquer le passé tout en généralisant mal sur des données financières futures.

5 Simulation d'un HMM et application à des données financières

5.1 Jeu de données simulé

Afin de tester et de valider les différents algorithmes étudiés, nous avons construit un jeu de données simulé à partir d'un modèle de Markov caché. L'objectif de cette simulation est de disposer d'une séquence d'observations pour laquelle les paramètres du modèle sont connus à l'avance. Cela permet ensuite de vérifier si les algorithmes d'inférence et d'estimation parviennent à retrouver correctement la structure sous-jacente du modèle.

Le modèle considéré possède trois états cachés, représentant trois conditions météorologiques possibles :

$$S = \{\text{soleil, nuageux, pluie}\}.$$

À chaque instant, l'état caché n'est pas observé directement. En revanche, on observe une variable liée à cet état, qui correspond ici au fait qu'une personne prenne ou non un parapluie :

$$O = \{\text{parapluie, pas parapluie}\}.$$

La distribution initiale des états est donnée par

$$\pi = (0.5 \quad 0.3 \quad 0.2),$$

ce qui signifie que la simulation commence avec une probabilité de 0.5 dans l'état « soleil », 0.3 dans l'état « nuageux » et 0.2 dans l'état « pluie ».

La matrice de transition entre les états cachés est définie par :

$$A = \begin{pmatrix} 0.7 & 0.295 & 0.005 \\ 0.3 & 0.4 & 0.3 \\ 0.005 & 0.395 & 0.6 \end{pmatrix}.$$

La matrice d'émission est quant à elle définie par :

$$B = \begin{pmatrix} 0.1 & 0.9 \\ 0.4 & 0.6 \\ 0.9 & 0.1 \end{pmatrix}.$$

À partir de ces paramètres, nous simulons une trajectoire de longueur $T = 1000$. La génération se fait en deux étapes à chaque instant. Tout d'abord, un état caché est tiré selon la matrice de transition A . Ensuite, une observation est générée selon la loi d'émission correspondant à cet état, donnée par la matrice B . On obtient ainsi deux suites :

$$(X_1, X_2, \dots, X_T)$$

pour les états cachés, et

$$(Y_1, Y_2, \dots, Y_T)$$

pour les observations.

Ce jeu de données simulé constitue un cadre pour évaluer les algorithmes. En effet, contrairement à un jeu de données réel, les états cachés et les vrais paramètres du modèle sont connus. Il est donc possible de comparer les résultats produits par les algorithmes avec les valeurs utilisées pour générer les données. Cela nous permettra notamment de tester la fiabilité des différents algorithmes.

5.2 Application des algorithmes sur données simulées

Résultat de l'algorithme Forward : -639.814329

Résultat de l'algorithme de Viterbi :

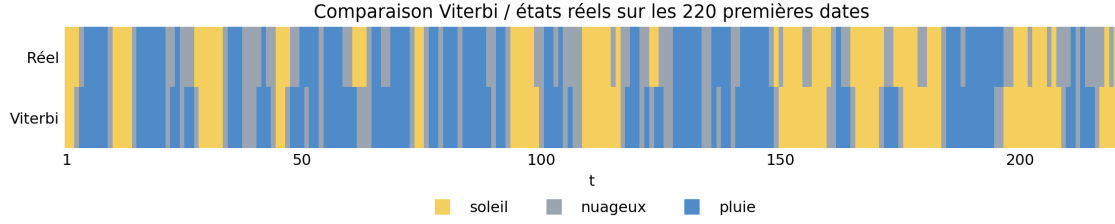


FIGURE 4 – Comparaison entre les états réels simulés et le chemin Viterbi sur les 220 premières dates.

Résultat de Baum-Welch : Après exécution de l'algorithme de Baum-Welch sur les données simulées, nous obtenons les paramètres estimés suivants.

La matrice de transition estimée est

$$\hat{A} = \begin{pmatrix} 0.7026 & 0.2956 & 0.0018 \\ 0.2720 & 0.5963 & 0.1316 \\ 0.1804 & 0.2172 & 0.6024 \end{pmatrix}.$$

La matrice d'émission estimée est

$$\hat{B} = \begin{pmatrix} 0.1070 & 0.8930 \\ 0.5702 & 0.4298 \\ 0.9895 & 0.0105 \end{pmatrix}.$$

5.3 Interprétation des résultats obtenus sur les données simulées

5.3.1 Analyse du problème de Forward

On applique les critères forward à la trajectoire simulée. Le modèle évalué est le modèle générateur $\lambda = (\pi, A, B)$.

Vraisemblance et log-vraisemblance

Critère	Valeur
Longueur de la série T	1001
Nombre de paramètres libres p	11
Vraisemblance brute	1.3557×10^{-278}
Log-vraisemblance	-639.814329
Log-vraisemblance moyenne	-0.639175
NLPD	0.639175
AIC indicatif	1301.628658
BIC indicatif	1355.624961

TABLE 1 – Critères forward sur la série complète.

La vraisemblance brute est très faible, ce qui est attendu pour une longue suite d’observations. On utilise donc la log-vraisemblance

$$\ell(\lambda) = \log P(Y_{1:T} \mid \lambda) = -639.814329.$$

La log-vraisemblance moyenne vaut

$$\bar{\ell}(\lambda) = \frac{1}{T}\ell(\lambda) = -0.639175,$$

et la negative log predictive density vaut

$$\text{NLPD} = -\bar{\ell}(\lambda) = 0.639175.$$

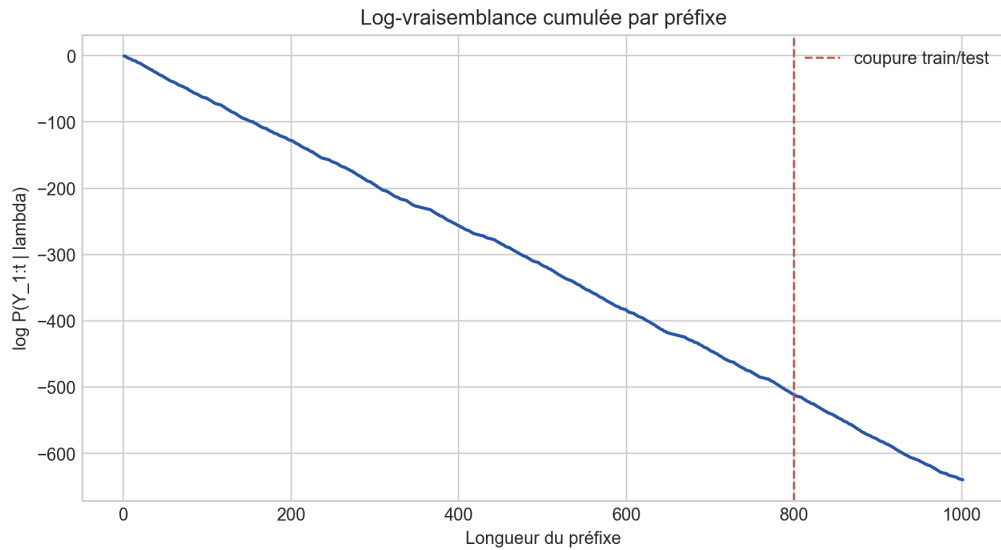


FIGURE 5 – Log-vraisemblance cumulée par préfixe.

Comparaison de modèles

On compare quatre modèles de même dimension :

$$\lambda_0 = (\pi, A, B), \quad \lambda_1 = (\pi, A_{\text{unif}}, B), \quad \lambda_2 = (\pi, A, B_{\text{unif}}), \quad \lambda_3 = (\pi_{\text{unif}}, A_{\text{unif}}, B_{\text{unif}}).$$

Le modèle λ_1 conserve les émissions mais rend les transitions uniformes. Le modèle λ_2 conserve les transitions mais rend les émissions uniformes. Le modèle λ_3 est entièrement uniforme.

Pour $N = 3$ états et $M = 2$ observations, on utilise

$$p = (N - 1) + N(N - 1) + N(M - 1) = 11.$$

Les critères sont

$$\text{AIC} = -2\ell(\lambda) + 2p, \quad \text{BIC} = -2\ell(\lambda) + p \log(T).$$

Modèle	log-vrais.	log-vrais./obs.	NLPD	AIC	BIC
Modèle de simulation	-639.814329	-0.639175	0.639175	1301.628658	1355.624961
Transitions uniformes	-689.395632	-0.688707	0.688707	1400.791264	1454.787566
Émissions uniformes	-693.840328	-0.693147	0.693147	1409.680655	1463.676958
Modèle uniforme	-693.840328	-0.693147	0.693147	1409.680655	1463.676958

TABLE 2 – Comparaison forward des modèles.

Le modèle de simulation minimise l'AIC et le BIC. Les modèles à émissions uniformes obtiennent une log-vraisemblance moyenne proche de $-\log(2)$, ce qui indique que supprimer le lien entre états cachés et observations dégrade fortement la qualité forward.

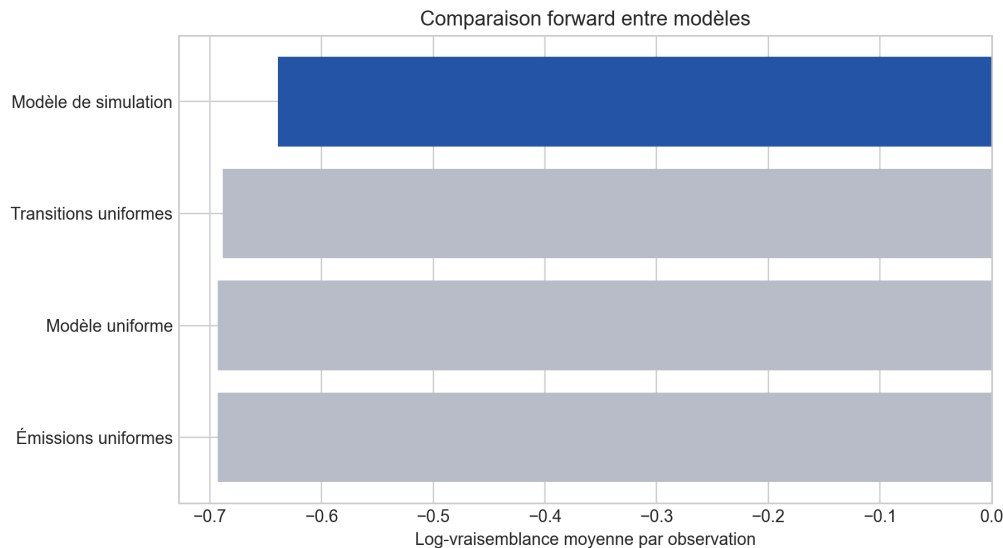


FIGURE 6 – Log-vraisemblance moyenne par modèle.

Validation hors échantillon

La série est découpée chronologiquement en 800 observations d'apprentissage et 201 observations de test. La performance hors échantillon est mesurée par :

$$\ell_{\text{test}} = \log P(Y_{\text{test}} | Y_{\text{train}}, \lambda), \quad \text{NLPD}_{\text{test}} = -\frac{1}{T_{\text{test}}} \ell_{\text{test}}.$$

Segment	T	log-vrais.	log-vrais./obs.	NLPD
Apprentissage	800	-511.319586	-0.639149	0.639149
Test conditionnel	201	-128.494743	-0.639277	0.639277
Test indépendant	201	-128.462530	-0.639117	0.639117

TABLE 3 – Validation forward hors échantillon.

La NLPD de test conditionnel vaut 0.639277, très proche de celle obtenue sur la partie d'apprentissage. Le modèle garde donc une qualité prédictive stable sur les observations de test.

Conclusion

Les critères forward donnent une évaluation cohérente : la log-vraisemblance moyenne vaut -0.639175, la NLPD vaut 0.639175, le modèle générateur minimise l'AIC et le BIC parmi les modèles comparés, et la NLPD hors échantillon reste proche de celle obtenue sur la période d'apprentissage. La comparaison avec les modèles uniformes montre en particulier que la matrice d'émission joue un rôle central : lorsque le lien entre états cachés et observations est supprimé, la vraisemblance se

dégrade nettement.

5.3.2 Analyse du problème de Viterbi

On applique les critères liés au décodage de Viterbi à la trajectoire simulée. Le chemin obtenu est noté $\hat{X}_{1:T}^V$.

Interprétation des régimes détectés

On commence par regarder si les états décodés correspondent à des profils d'observations distincts.

État Viterbi	Dates	Parapluie obs.	Parapluie théorique	Pas parapluie obs.
soleil	408	9.56%	10.00%	90.44%
nuageux	206	12.62%	40.00%	87.38%
pluie	387	100.00%	90.00%	0.00%

TABLE 4 – Profil des observations dans chaque état décodé par Viterbi.

Les états *soleil* et *pluie* sont bien séparés : l'état *soleil* est associé presque toujours à *Pas parapluie*, tandis que l'état *pluie* est associé presque toujours à *Parapluie*. L'état *nuageux* est moins clairement identifié, ce qui est cohérent avec des probabilités d'émission moins extrêmes.

On regarde ensuite la persistance des régimes.

Séquence	Changements	Taux	Runs	Longueur moy.	Longueur max
Viterbi	326	32.60%	327	3.06	28
Posterior decoding	390	39.00%	391	2.56	21
Décodage local émissions	358	35.80%	359	2.79	23

TABLE 5 – Persistance des séquences décodées.

Le chemin Viterbi effectue 326 changements, soit un taux de 32.60%. Il est plus régulier que le posterior decoding, ce qui correspond à son objectif : trouver une trajectoire globale cohérente.

Comparaison avec le posterior decoding

Le posterior decoding choisit, à chaque date, l'état marginalement le plus probable :

$$\hat{X}_t^P \in \arg \max_i P(X_t = i \mid Y_{1:T}, \lambda).$$

On compare les deux décodages avec le taux de désaccord

$$D_{VP} = \frac{1}{T} \sum_{t=1}^T \mathbf{1}_{\{\hat{X}_t^V \neq \hat{X}_t^P\}}.$$

Critère	Valeur
D_{VP}	8.39%
Désaccord Viterbi vs local émissions	24.48%
Proba postérieure moyenne de l'état Viterbi	66.04%
Proba postérieure médiane de l'état Viterbi	67.99%
Marge postérieure moyenne	38.33%
Marge postérieure médiane	42.12%

TABLE 6 – Comparaison entre Viterbi et posterior decoding.

On obtient $D_{VP} = 8.39\%$. Les deux méthodes donnent donc des classifications proches. La probabilité postérieure moyenne de l'état choisi par Viterbi vaut 66.04%, ce qui indique que le chemin Viterbi suit le plus souvent des états marginalement plausibles.

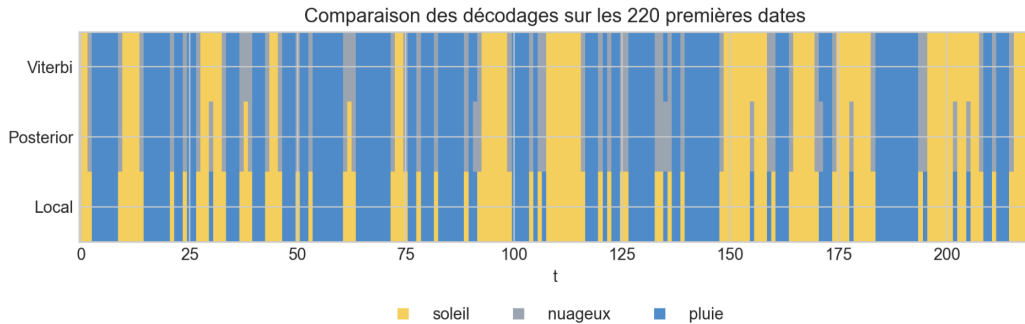


FIGURE 7 – Comparaison visuelle entre Viterbi, posterior decoding et décodage local.

Conclusion

Les critères donnent une lecture cohérente du décodage. Les régimes extrêmes sont bien interprétables à partir des observations, et le taux de désaccord avec le posterior decoding reste faible. Le chemin Viterbi est aussi plus régulier que les décodages fondés sur des décisions locales. Le régime *nuageux* reste moins nettement séparé, ce qui est cohérent avec des probabilités d'émission moins discriminantes.

5.3.3 Analyse du problème de Baum-Welch

Nous avons appliqué l'algorithme de Baum-Welch au jeu de données simulé présenté précédemment, en lançant l'estimation à partir de plusieurs initialisations différentes. L'objectif est ici d'observer si les paramètres estimés sont stables d'une initialisation à l'autre et si les solutions obtenues conduisent à des vraisemblances comparables.

La Figure 8 compare les log-vraisemblances finales obtenues pour les différentes initialisations. On observe que les écarts de log-vraisemblance par observation sont très faibles, de l'ordre de 10^{-4} à 10^{-3} . Cela signifie que, du point de vue de la vraisemblance, les différentes estimations expliquent les observations de manière assez proche. Certaines initialisations, notamment les initialisations 9 et 10, donnent des résultats quasiment identiques à la meilleure solution. À l'inverse, l'initialisation 8 présente l'écart le plus important, ce qui indique une solution légèrement moins bonne, même si l'écart reste faible une fois rapporté au nombre d'observations.

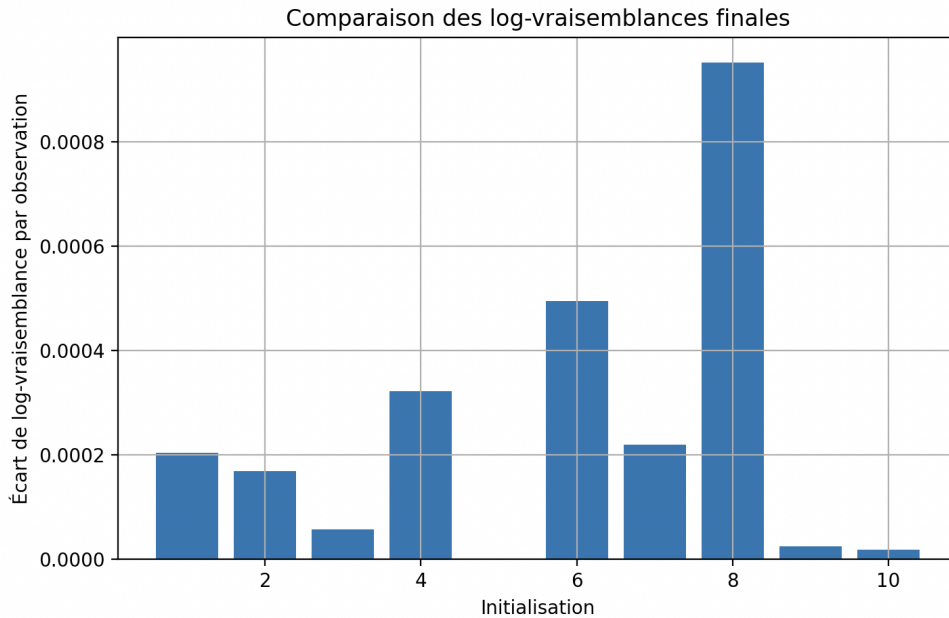


FIGURE 8 – Comparaison des log-vraisemblances finales selon l'initialisation.

Les Figures 9 et 10 montrent respectivement la stabilité de la matrice de transition A et de la matrice d'émission B . Ces deux matrices présentent des écarts plus marqués selon l'initialisation. On observe notamment que les initialisations 1, 2, 6, 7 et 8 conduisent à des distances relativement importantes par rapport à la meilleure solution. Cela montre que les paramètres estimés ne sont pas parfaitement stables.

Ce résultat est cohérent avec le comportement attendu de Baum-Welch : l'algorithme peut converger vers des maxima locaux différents selon le point de départ. Ainsi, deux initialisations peuvent donner des log-vraisemblances finales proches tout en produisant des matrices A et B différentes. Autrement

dit, la vraisemblance seule ne suffit pas toujours à garantir que les régimes estimés ont exactement la même interprétation.

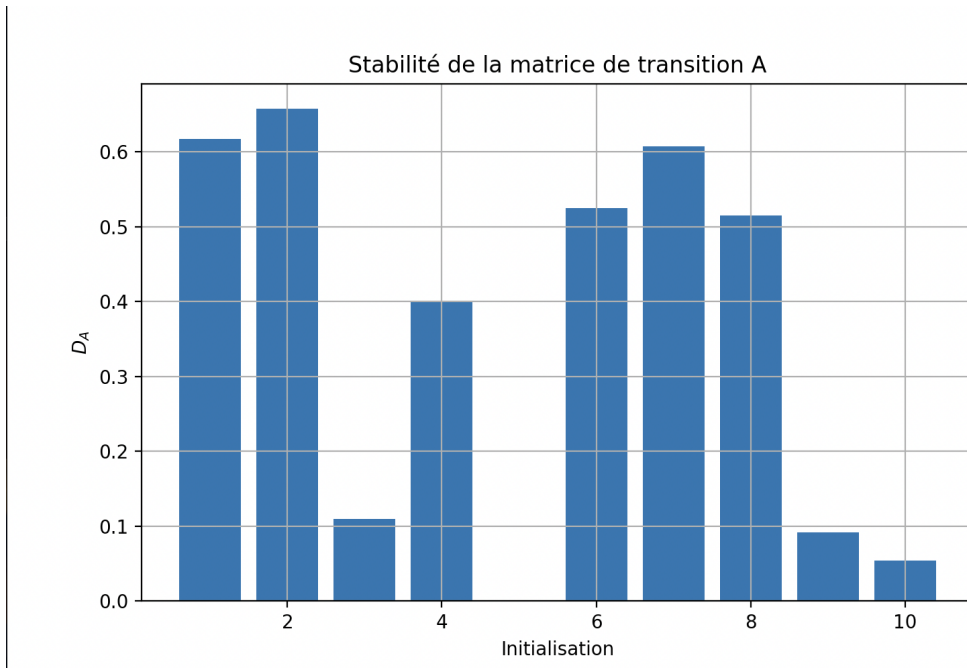


FIGURE 9 – Stabilité de la matrice de transition estimée A .

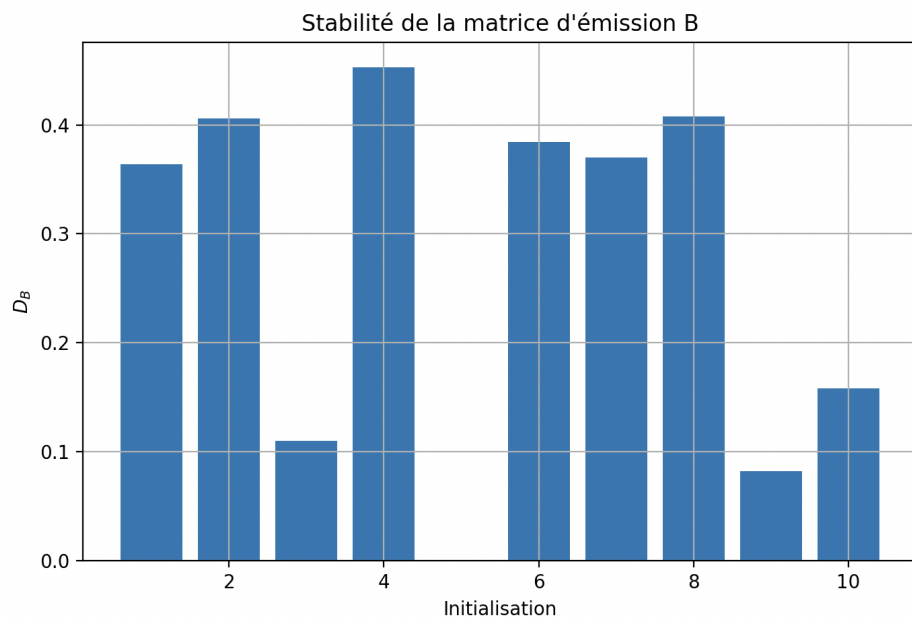


FIGURE 10 – Stabilité de la matrice d'émission estimée B .

La Figure 11 permet d'interpréter la stabilité de la matrice de transition sous un angle plus concret,

en comparant les durées moyennes des régimes. On remarque que les initialisations pour lesquelles la matrice A est éloignée de la référence sont aussi celles pour lesquelles les durées moyennes des régimes varient le plus fortement. Les initialisations 1, 2, 6, 7 et 8 présentent ainsi des écarts importants, tandis que les initialisations 3, 9 et 10 sont beaucoup plus proches de la solution de référence.

Cela signifie que certaines estimations ne décrivent pas seulement des matrices différentes : elles conduisent également à une interprétation différente de la persistance des régimes cachés. Par exemple, un régime peut être estimé comme plus stable ou plus transitoire selon l'initialisation. Cette observation est importante, car dans un modèle de Markov caché, la durée moyenne passée dans chaque état est une information essentielle pour interpréter les régimes.

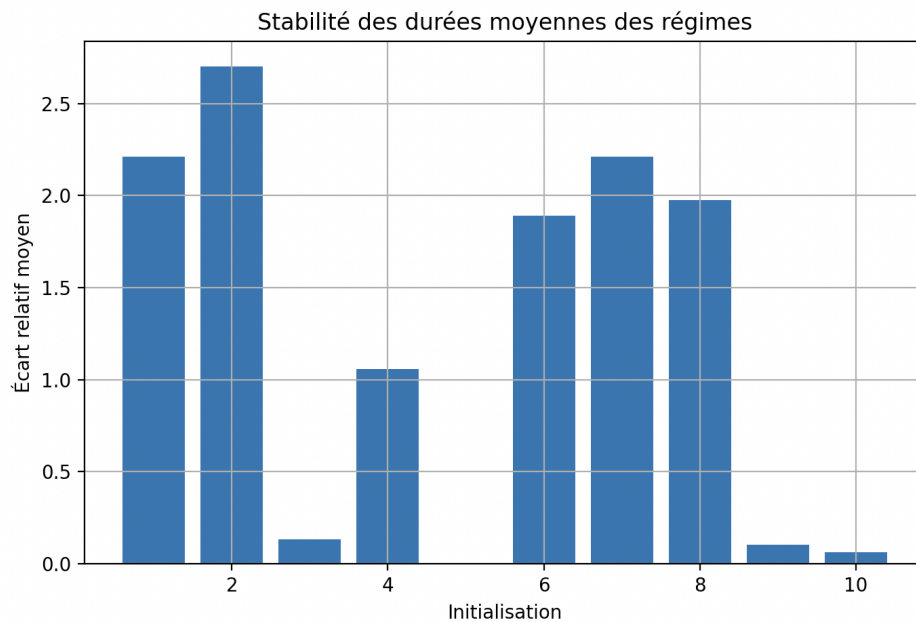


FIGURE 11 – Stabilité des durées moyennes des régimes.

Globalement, les résultats montrent que l'algorithme parvient à obtenir des solutions proches en termes de vraisemblance, mais que les paramètres estimés peuvent varier sensiblement selon l'initialisation. Cette différence est particulièrement visible sur la matrice de transition A , la matrice d'émission B et les durées moyennes des régimes. Cela confirme l'intérêt de lancer Baum-Welch plusieurs fois avec des initialisations différentes, puis de retenir la solution ayant la meilleure log-vraisemblance finale.

Dans notre cas, les initialisations 9 et 10 semblent très proches de la solution de référence, à la fois en termes de vraisemblance et de paramètres estimés. À l'inverse, certaines initialisations donnent des paramètres plus éloignés malgré une vraisemblance finale relativement proche. On peut donc conclure que l'algorithme fonctionne correctement sur les données simulées, mais que son résultat reste sensible à l'initialisation, ce qui constitue une limite classique des méthodes de type EM.

5.4 Détection de régimes de volatilité sur le VIX

Dans cette section, nous appliquons un modèle de Markov caché au VIX, c'est-à-dire à l'indice de volatilité implicite du S&P 500. L'objectif est d'étudier si un modèle HMM permet d'identifier automatiquement différents régimes de marché à partir de l'évolution de la volatilité. En effet, le VIX est souvent interprété comme un indicateur du niveau de stress ou d'incertitude sur les marchés financiers. Un niveau faible du VIX correspond généralement à une période calme, tandis qu'un niveau élevé est associé à des phases de tension ou de crise.

Les données utilisées sont des observations journalières du VIX entre 2004 et 2024. Après importation des données, nous conservons principalement le prix de clôture journalier du VIX, noté V_t . Afin d'éviter de construire le modèle directement sur le niveau brut du VIX, nous introduisons plusieurs variables décrivant son comportement dynamique. On considère d'abord le logarithme du VIX :

$$X_t = \log(V_t),$$

puis le rendement logarithmique journalier :

$$R_t = X_t - X_{t-1}.$$

Nous calculons également deux mesures de volatilité réalisée, obtenues comme écart-type glissant des rendements logarithmiques :

$$\sigma_t^{(20)} = \text{std}(R_{t-19}, \dots, R_t),$$

et

$$\sigma_t^{(5)} = \text{std}(R_{t-4}, \dots, R_t).$$

La première mesure donne une estimation de la volatilité réalisée sur environ un mois de cotation, tandis que la seconde capture une dynamique plus courte et plus réactive.

Nous utilisons donc comme variables observées le vecteur

$$Y_t = \begin{pmatrix} R_t \\ \sigma_t^{(20)} \\ \sigma_t^{(5)} \end{pmatrix}.$$

Ces variables sont ensuite standardisées avant l'estimation du modèle, afin que les différentes composantes aient des ordres de grandeur comparables.

On considère un modèle de Markov caché à trois états cachés :

$$S_t \in \{0, 1, 2\}.$$

Conditionnellement à l'état caché $S_t = i$, l'observation Y_t est supposée suivre une loi gaussienne

multivariée :

$$Y_t \mid S_t = i \sim \mathcal{N}(\mu_i, \Sigma_i),$$

où μ_i est le vecteur moyen associé au régime i et Σ_i sa matrice de covariance.

L'estimation du modèle est réalisée à l'aide de l'algorithme de Baum-Welch. Une fois le modèle estimé, la séquence la plus probable des états cachés est obtenue par l'algorithme de Viterbi.

Dans notre application, nous avons choisi trois régimes afin d'obtenir une interprétation simple des phases de marché :

régime calme, régime intermédiaire, régime de crise.

Les états obtenus par le HMM n'ayant pas d'ordre naturel, nous les réordonnons ensuite selon la volatilité réalisée moyenne $\sigma_t^{(20)}$. Le régime ayant la plus faible volatilité moyenne est appelé régime calme, celui ayant une volatilité moyenne intermédiaire est appelé régime intermédiaire, et celui ayant la volatilité moyenne la plus élevée est appelé régime de crise.

Les statistiques moyennes obtenues pour les trois régimes sont les suivantes :

Régime	VIX moyen	R_t moyen	$\sigma_t^{(20)}$ moyenne	Nombre d'observations
Calme	16.3809	0.0007	0.0466	2364
Intermédiaire	20.0225	-0.0021	0.0719	1751
Crise	23.6703	0.0018	0.1134	1003

La matrice de transition estimée, après réordonnancement des états, est donnée par :

$$\hat{A} = \begin{pmatrix} 0.972 & 0.020 & 0.007 \\ 0.038 & 0.941 & 0.021 \\ 0.000 & 0.053 & 0.947 \end{pmatrix}.$$

On observe que les coefficients diagonaux sont élevés. Cela signifie que les régimes sont persistants. La figure suivante représente le VIX sur la période étudiée, avec une coloration de fond indiquant le régime estimé par le modèle HMM.

On remarque que le modèle identifie majoritairement des phases calmes, associées au régime de plus faible volatilité, tout en basculant vers les régimes intermédiaire et de crise lors des épisodes de forte instabilité. Ces changements apparaissent notamment autour des grandes hausses du VIX, comme pendant la crise financière de 2008, la crise de la dette souveraine en 2011–2012, ainsi que le choc de volatilité observé en 2020. Le modèle semble donc capable de repérer automatiquement des périodes de stress de marché à partir des variables construites sur le VIX.

Il faut toutefois souligner que les régimes estimés ne possèdent pas directement une interprétation

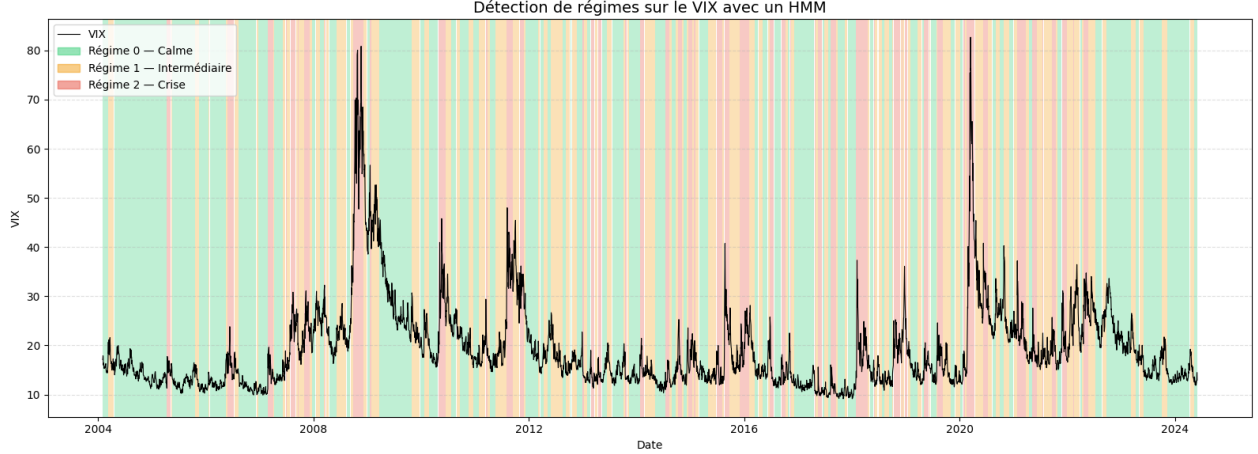


FIGURE 12 – Détection de régime sur l'indice du VIX entre 2004 et 2024

économique. Le HMM classe les observations selon leurs propriétés statistiques, en particulier les rendements et les niveaux de volatilité réalisée, mais il n'associe pas lui-même un état caché à un événement économique précis. L'interprétation des régimes doit donc être faite a posteriori, en comparant les périodes détectées avec les épisodes connus de tension sur les marchés.

Les résultats dépendent également des choix de modélisation, notamment des variables utilisées, du nombre d'états cachés et de l'initialisation de l'algorithme. Le choix de trois régimes offre ici une lecture naturelle mais d'autres spécifications pourraient conduire à une segmentation différente.

Cette application illustre ainsi l'intérêt des modèles de Markov cachés pour l'analyse de séries financières. Dans le cas du VIX, le HMM fournit une méthode probabiliste simple pour classifier les périodes de marché selon leur niveau de volatilité et leur dynamique de persistance.

5.5 Analyse de la qualité de l'estimation Baum–Welch sur le VIX

On applique les critères d'évaluation de Baum–Welch au modèle HMM estimé sur le VIX. Les observations utilisées sont les mêmes que précédemment :

$$Y_t = \begin{pmatrix} R_t \\ \sigma_t^{(20)} \\ \sigma_t^{(5)} \end{pmatrix},$$

où R_t est le rendement logarithmique journalier du VIX, $\sigma_t^{(20)}$ la volatilité réalisée sur vingt jours et $\sigma_t^{(5)}$ la volatilité réalisée sur cinq jours. Les variables sont standardisées avant l'estimation. Le modèle possède trois états cachés et des émissions gaussiennes multivariées.

La série utilisée contient 5118 observations, du 2 février 2004 au 31 mai 2024.

Convergence de la log-vraisemblance

Baum–Welch est lancé avec plusieurs initialisations. La meilleure solution est obtenue à la dixième initialisation. La log-vraisemblance finale vaut

$$\ell(\hat{\lambda}) = -14300.087178,$$

soit une log-vraisemblance moyenne de

$$\frac{\ell(\hat{\lambda})}{T} = -2.794077.$$

Critère	Valeur
Nombre d’observations T	5118
Meilleure initialisation	10
Nombre d’itérations	36
Log-vraisemblance finale	-14300.087178
Log-vraisemblance moyenne	-2.794077
Gain final par observation	9.728857×10^{-11}
Gain relatif final	3.481714×10^{-11}
Nombre de baisses détectées	0

TABLE 7 – Convergence de Baum–Welch sur le VIX.

La log-vraisemblance est croissante et aucune baisse numérique n’est détectée. Le gain final par observation est très inférieur à 10^{-6} , ce qui indique que l’algorithme s’est stabilisé. La convergence est donc satisfaisante.

Stabilité des paramètres estimés

Comme Baum–Welch dépend de l’initialisation, l’estimation est répétée dix fois. Les états sont ensuite alignés afin de corriger le changement d’étiquette. Pour les émissions, le critère discret D_B est remplacé par une distance gaussienne fondée sur une divergence de Kullback–Leibler symétrisée entre les lois d’émission.

Critère de stabilité	Valeur moyenne
D_π	$2.563015 \times 10^{-109}$
D_A	5.050970×10^{-7}
D_B gaussien	7.307790×10^{-10}
Écart relatif moyen des durées	1.257620×10^{-5}
Écart maximal de log-vraisemblance par observation	1.587589×10^{-10}

TABLE 8 – Stabilité des estimations selon l’initialisation.

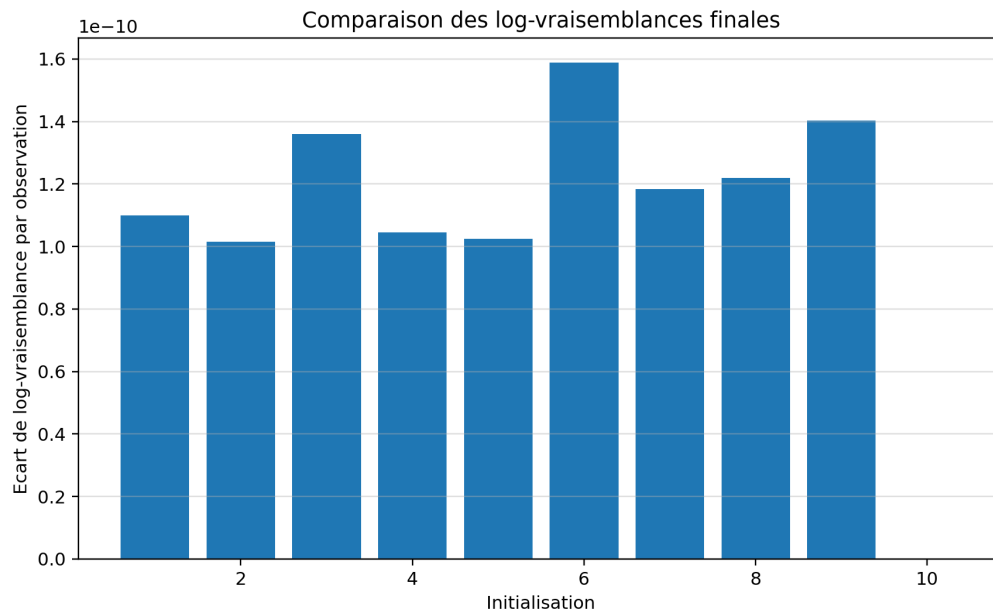


FIGURE 13 – Comparaison des log-vraisemblances finales.

Les écarts entre initialisations sont extrêmement faibles. Les log-vraisemblances finales sont presque identiques, les matrices de transition sont très proches et les émissions gaussiennes restent stables après alignement des états. Contrairement au cas où Baum–Welch converge vers plusieurs optima locaux distincts, les résultats obtenus ici indiquent une solution très stable pour ce choix de variables et de nombre d’états.

Validation hors échantillon

La série est découpée chronologiquement : 4094 observations servent à l’apprentissage et 1024 observations servent au test. Le modèle est estimé sur la période d’apprentissage puis évalué sur la période suivante.

Segment	T	Log-vraisemblance	NLPD
Apprentissage	4094	-11536.441391	2.817890
Test conditionnel	1024	-2774.008272	2.708992
Test indépendant	1024	-2782.320895	2.717110

TABLE 9 – Validation hors échantillon du modèle HMM sur le VIX.

La NLPD de test conditionnel vaut 2.708992, contre 2.817890 sur la période d'apprentissage. Le modèle ne présente donc pas de dégradation hors échantillon sur cette découpe chronologique. Cette comparaison ne signifie pas que la période de test est toujours plus facile à prévoir, mais elle indique que les paramètres estimés sur la première partie de la série conservent une bonne capacité à attribuer une densité élevée aux observations futures.

Conclusion

Les critères appliqués au VIX donnent une évaluation favorable de l'estimation Baum–Welch. La log-vraisemblance converge sans baisse numérique, les différentes initialisations conduisent pratiquement à la même solution après alignement des états, et les régimes obtenus sont interprétables en termes de volatilité de marché. La validation hors échantillon confirme que le modèle reste stable sur la période de test. La principale différence avec la simulation est qu'il n'existe pas ici de vrais états cachés observables : l'évaluation repose donc sur la convergence, la stabilité des paramètres, la cohérence économique des régimes et la performance prédictive.

6 Références

Références

- [1] Rabiner, L. R. (1989). *A tutorial on hidden Markov models and selected applications in speech recognition*. <https://www.cs.ubc.ca/~murphyk/Bayes/rabiner.pdf>
- [2] Jurafsky, D., & Martin, J. H. (2026). *Hidden Markov Models*. In *Speech and Language Processing* (Chapter A, draft of January 6, 2026). Stanford University. <https://web.stanford.edu/~jurafsky/slp3/A.pdf>
- [3] Sublime, J. (2022). *Data Analysis – Lecture 9 : Time series analysis, Hidden Markov Models*. Engineering school lecture notes, France. HAL, hal-03845332. <https://hal.science/hal-03845332/document>
- [4] Baum, L. E., & Petrie, T. (1966). *Statistical inference for probabilistic functions of finite state Markov chains*. The Annals of Mathematical Statistics, 37(6), 1554–1563. <https://doi.org/10.1214/aoms/1177699147>

- [5] Baum, L. E., Petrie, T., Soules, G., & Weiss, N. (1970). *A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains*. The Annals of Mathematical Statistics, 41(1), 164–171. <https://doi.org/10.1214/aoms/1177697196>
- [6] Wu, C. F. J. (1983). *On the convergence properties of the EM algorithm*. The Annals of Statistics, 11(1), 95–103. <https://doi.org/10.1214/aos/1176346060>
- [7] Leroux, B. G. (1992). *Maximum-likelihood estimation for hidden Markov models*. Stochastic Processes and their Applications, 40(1), 127–143. [https://doi.org/10.1016/0304-4149\(92\)90141-C](https://doi.org/10.1016/0304-4149(92)90141-C)
- [8] Lember, J., & Koloydenko, A. A. (2014). *Bridging Viterbi and posterior decoding : A generalized risk approach to hidden path inference based on hidden Markov models*. Journal of Machine Learning Research, 15, 1–58. <https://jmlr.org/papers/v15/lemb14a.html>
- [9] Akaike, H. (1974). *A New Look at the Statistical Model Identification*. In E. Parzen, K. Tanabe, & G. Kitagawa (Eds.), *Selected Papers of Hirotugu Akaike* (pp. 215–222). Springer Series in Statistics. Springer, New York, NY. <https://doi.org/10.1109/TAC.1974.1100705>